

BSS Machine — Gate1 / Gate2 Architecture

(Design Snapshot)

Purpose (one line): An iterative selection-control machine that alternates a local *pull* (k-nearest + gradient descent) gate and a global *push* (k-farthest / dispersion + gradient ascent) gate, with global exponentiation to sharpen or smooth selections — intended as a differentiable, end-to-end module (relaxed discrete selection) suitable for later optimization and integration.

1. Core idea

Operate on a finite dataset $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^d$ with a continuous selector vector $s \in \mathbb{R}_+^n$ ($\sum_i s_i = k$) and an optional probe/query $q \in \mathbb{R}^d$. Alternate two complementary gates:

- **Gate 1 — Pull (atomic recursion):** concentrates mass on points locally close to a probe or local attractor (k-nearest behaviour implemented by differentiable soft selection and gradient descent on the probe or selection weights).
- **Gate 2 — Push (k-farthest / dispersion):** re-distributes mass to emphasize global dispersion or far-apart choices (maximize pairwise distances or related surrogates via gradient ascent on the relaxed selector).

A **global exponentiation** step (elementwise power followed by renormalization) sharpens or flattens the selector to control exploration↔exploitation and progressively induce discrete k-hot selections.

2. Notation & primitives

- Dataset: $X = [x_1, \dots, x_n], x_i \in \mathbb{R}^d$.
 - Distance matrix: $D_{ij} = \|x_i - x_j\|^2$ (or another metric).
 - Selector: $s \in \mathbb{R}_{\geq 0}^n$ with $\mathbf{1}^\top s = k$. When discrete, s is k-hot.
 - Probe/query: $q \in \mathbb{R}^d$ (optional; used by Gate1 for attraction).
 - Objectives:
 - Pull objective (minimize): $L_{pull}(q, s) = \sum_i s_i \|q - x_i\|^2$ or use softmax over distances to define weights.
 - Push objective (maximize): $L_{push}(s) = \frac{1}{2} s^\top D s$ (sum of pairwise distances) or smoothed k-center surrogate.
 - Projection: projection onto the simplex $\Delta_k = \{s \geq 0, \sum s = k\}$ via standard simplex projection.
-

3. Gate descriptions (essentials)

Gate 1 — Pull (atomic recursion)

Inputs: current probe q , selector s (optional). Operation patterns: 1. **Probe update:** compute soft neighborhood weights $w_i \propto \exp(-\|q - x_i\|^2 / \tau)$ and update probe by weighted mean: $q \leftarrow \sum_i w_i x_i$

or gradient descent on L_{pull} . 2. **Selector tightening (optional)**: move selector mass toward nearest points: $s \leftarrow \text{proj}_{\Delta_k}(s - \eta_1 d(q, X))$ (descent on distances). Output: updated q and/or s concentrated near local region.

Rationale: Gate1 models attraction/recursion — it finds local modes and gathers mass around dense neighborhoods.

Gate 2 — Push (k-farthest / dispersion)

Inputs: selector s . Core operation (continuous relaxation): - Compute gradient $g = Ds$ for $L_{push}(s) = \frac{1}{2} s^\top Ds$. - Ascend: $s \leftarrow s + \eta_2 g$ then $s \leftarrow \text{proj}_{\Delta_k}(s)$. Optional surrogate: maximize smoothed min pairwise distance using log-sum-exp approximations if strict k-center behaviour is desired. Output: selector s redistributed to emphasize global spread.

Rationale: Gate2 forces diversity / exploration and prevents collapse to a single mode.

4. Global exponentiation

Apply elementwise power: $s \leftarrow s^\gamma$ then renormalize to sum k. Behavior: - $\gamma > 1$: sharpens (amplify large entries) $\rightarrow$ moves toward discrete selection. - $0 < \gamma < 1$: smooths (flatten differences) $\rightarrow$ encourages exploration.

Use annealing schedules on γ (e.g. slowly increase from 1 to a max) to transition from soft exploration to discrete exploitation.

5. Iterative control loop (concise)

1. Initialize: $s = (k/n)\mathbf{1}$, $q = \text{mean}(X)$, $\gamma = \gamma_0$.
2. Repeat until stop:
3. Gate1: compute soft neighborhood, update q (or update s toward near points).
4. Gate2: gradient ascent on dispersion, project to simplex.
5. Exponentiate: $s \leftarrow s^\gamma$, renormalize.
6. Anneal: update γ (and optionally learning rates).
7. Extract discrete set: top-k indices of s (optionally refine by local discrete swap search).

6. Data structures & compute footprint (explicit/basic form)

- **Distance matrix** D : full $n \times n$ dense ($O(n^2)$ memory/time) — baseline design; easy gradients and exact objectives.
- **Selector** s : dense vector $\mathbb{R}^n$.
- **Probe** q : $\mathbb{R}^d$.

This design intentionally accepts the dense D for clarity and exactness; later optimizations can replace it with sparse k-NN graphs, ANN, or low-rank kernels.

7. Interfaces & outputs

- **Inputs:** dataset X , k , initial hyperparameters $(\eta_1, \eta_2, \gamma_0, \tau, \text{max_iter})$.
 - **Primary outputs:** selector s (soft) and discrete set $S = \text{top-}k(s)$.
 - **Diagnostics:** iteration history of objectives (pull and push), selector entropy, probe trajectories, convergence flags.
-

8. Hyperparameter suggestions (starting defaults)

- η_1, η_2 : 0.01 — 0.1 (tune by scale of D). Use momentum or Adam optionally.
 - $\gamma_0 = 1.0$, ramp to $\gamma_{max} = 8-16$ multiplicatively.
 - τ (Gate1 softmax temp): median squared distance of X .
 - Iterations: 50–500 (depends on n , desired sharpness).
-

9. Failure modes & quick mitigations

- **Collapse to single cluster (lack of diversity):** increase γ slower, boost push learning rate, or initialize multiple probes.
 - **Premature sharpness / poor local optima:** use multiple restarts and gradual annealing.
 - **Computational cost (n large):** later replace dense D with approximate/sparse methods or hierarchical two-stage selection.
-

10. Extensions & next steps (brief)

- Replace full D with sparse k -NN graphs or low-rank approximations for scale.
 - Add entropy or repulsive regularizers to stabilize continuous dynamics.
 - Parallelize multiple probes q_j and merge via Gate2 to obtain multi-modal representatives.
 - Provide a discrete polish stage (swap-based local search) for guaranteed improvement of the final discrete set.
-

Status: concise, implementable blueprint (dense baseline). This document prioritizes clarity and completeness of the essential mechanism; optimizations and scaling strategies are deliberately deferred to later refinement stages.